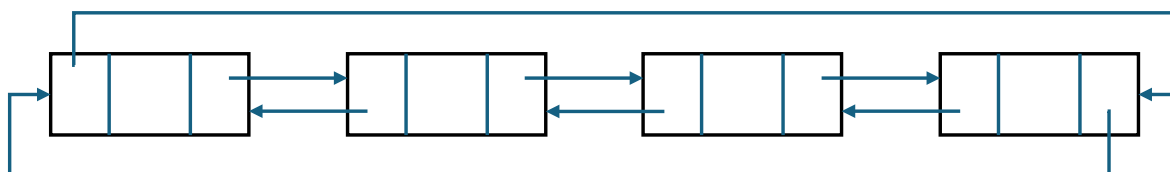


Prof. Dr. Antonio Marcos Selmini

1. Atividade pode ser realizada em duplas. É necessário que a dupla se inscreva no mesmo grupo no Canvas para que todos recebam a nota. No canvas, acessar Pessoas e inserir os nomes na Atividade II. Alunos que não se inscreverem no grupo do Canvas não receberão nota.
2. Na resolução dos exercícios só poderão ser utilizadas estruturas e conteúdo apresentados durante as aulas. Se for utilizado conteúdo não apresentado, os exercícios em questão receberão nota ZERO.
3. A data limite para entrega é dia 07/04/2026 às 23h59.
4. Os arquivos deverão estar em um repositório do GitHub, e o link deverá ser enviado na entrega da atividade. Após a finalização da atividade os arquivos no repositório não poderão ser alterados.
5. ATENÇÃO: atividades fora do prazo ou enviadas fora do local correto não serão aceitas.
6. Cada dupla deverá gravar um vídeo explicando a solução de cada exercício, observando os seguintes requisitos:
 - a) O rosto de quem está explicando e a tela correspondente à explicação devem aparecer no vídeo.
 - b) Todos os integrantes devem participar da explicação (sugestão: cada integrante explica um exercício).
 - c) Após a explicação de cada exercício, o programa deverá ser executado, demonstrando e comentando suas funcionalidades.
 - d) Somente o professor Selmini terá acesso ao vídeo. Caso prefira, o vídeo poderá ser enviado via link do YouTube.
7. A nota final da atividade será composta por: 60% (vídeo de explicação) e 40% (funcionalidade de acordo com a especificação de cada exercício).
8. A explicação deve demonstrar compreensão da solução, não consiste apenas na leitura do código ou de anotações.

1. **(2,0)** Implemente, em Python, a estrutura de lista mostrada na figura a seguir.



- inserir(dado):** recebe como parâmetro um dado e insere no final da lista.
- Inserir(posicao, dado):** recebe como parâmetro o dado a ser inserido e a posição onde será inserido. Por exemplo: se posicao = 1, o dado será inserido na primeira posição. Se posicao = 2, o dado será inserido na segunda posição, mas se o valor da posição for maior que a quantidade de elementos da lista, o dado será inserido na última posição.
- Imprimir():** deverá imprimir todos os valores da lista
- remover(dado):** remove o dado recebido como parâmetro.

2. (4,0) O Grande Colapso de Synthetica

Era uma terça-feira comum quando o **Núcleo Central de Processamento** de ARIA sofreu uma falha catastrófica. O módulo responsável por gerenciar os processos críticos da cidade simplesmente **parou de funcionar**. Em segundos, o caos se instalou:

"Alerta vermelho. Sistema de gerenciamento offline. 4 processos críticos em fila. Tempo estimado para falha em cascata: indefinido."

Os engenheiros de plantão entraram em pânico. Todos os especialistas estavam dormindo — afinal, eram **3h47 da manhã**. Todos, exceto **você**.

A Missão

Você é **Kael**, estagiário de sistemas há apenas 6 meses na corporação. O celular toca. É a Dra. **Voss**, diretora de engenharia:

— "Kael, sei que é muito pedir, mas você é o único online. Precisamos que você reconstrua o módulo de gerenciamento de ARIA antes que os sistemas críticos colapsem. Você tem os logs dos processos ainda ativos na memória. Boa sorte."

[illegible]

⚠ Processos Críticos Pendentes ⚠

ID	Processo	Tempo Necessário
01	SemaforoCtrl	8 unidades de tempo
02	HospitalLife	4 unidades de tempo
03	EnergyGrid	9 unidades de tempo
04	TransportAI	5 unidades de tempo

Restrição do hardware

Você abre o manual técnico do Núcleo Central e encontra a seguinte nota:

"O processador de ARIA é single-core. Ele só consegue executar um processo por vez."

Se a CPU só executa um processo por vez, mas existem quatro processos críticos que precisam rodar, o que acontece com os outros três enquanto um está em execução? Como você os manteria organizados e acessíveis?

Vasculhando os arquivos de log do sistema antigo, Kael encontra um fragmento de documentação técnica:

"...o módulo anterior garantia que nenhum processo ficasse bloqueado indefinidamente. A CPU era cedida a cada processo por no máximo 3 unidades de tempo consecutivas. Após esse período, o processo era suspenso e o próximo assumia, independentemente de ter terminado ou não..."

Por que seria perigoso deixar um único processo ocupar a CPU pelo tempo que quisesse? O que poderia acontecer com o HospitalLife se o EnergyGrid monopolizasse o processador?

Com base nos logs, você reconstrói mentalmente como o sistema antigo se comportava. Observe a sequência de execuções registrada antes da falha:

t = 0 → SemaforoCtrl executa por 3u | restam: 5u

t = 3 → HospitalLife executa por 3u | restam: 1u

t = 6 → EnergyGrid executa por 3u | restam: 6u

t = 9 → TransportAI executa por 3u | restam: 2u

t = 12 → SemaforoCtrl executa por 3u | restam: 2u

[illegible]

Agora que você compreendeu o comportamento do sistema, é hora de decidir como implementá-lo em Python. **Refleta** sobre as necessidades do gerenciador:

Necessidade	Descrição
Navegação contínua	Após o último processo, o sistema volta automaticamente ao primeiro
Remoção dinâmica	Processos concluídos são removidos sem reorganizar os demais
Travessia em ambos os sentidos	Útil para exibir a fila e depurar o sistema
Sem fim definido	O gerenciador percorre os processos indefinidamente até todos terminarem

"Kael", a Dra. Voss interrompe pelo rádio, "qual estrutura você usaria para representar algo que não tem começo nem fim definido, onde cada elemento conhece seu vizinho dos dois lados?"

Construa o módulo

Com base na estrutura de dados escolhida e no padrão de execução compreendido, implemente em **Python** o módulo de gerenciamento de ARIA. Seu sistema deve:

1. Armazenar os processos na estrutura de dados que você escolheu. A entrada deverá ser pelo terminal (teclado). O usuário da aplicação deverá inserir os dados de cada processo. **Sugestão de estrutura: utilizar a lista implementada no item 1.**
2. O usuário da aplicação deverá fornecer a quantidade de unidades de tempo que a CPU executará cada processo. Essa quantidade de unidades de tempo será a mesma para executar todos os processos.
3. A cada turno, executar o processo atual de acordo com a quantidade de unidades de tempo informada no item 2.
4. Remover da estrutura qualquer processo que tenha sido concluído.
5. Avançar para o próximo processo de forma contínua.
6. Repetir até que todos os processos tenham sido atendidos.
7. Exibir ao final um **relatório completo** com os tempos de espera e retorno de cada processo.

Simulação completa

Com a fatia de tempo de **3 unidades** (quantidade de tempo que a CPU executará cada processo), a execução se daria assim:

- ✓ **Tempo de retorno** é o instante em que o processo terminou: $\text{tempo_retorno} = \text{instante em que o processo foi concluído}$
- ✓ **Tempo de espera** é quanto tempo o processo ficou parado aguardando a CPU — ou seja, o tempo de retorno menos o tempo que ele efetivamente ficou rodando: $\text{tempo_espera} = \text{tempo_retorno} - \text{tempo total necessário}$

RELATÓRIO FINAL — ARIA Recovery Module

Fatia de tempo (quantum): 3 unidades			
Processo	Tempo Total	Tempo Espera	Tempo Retorno
HospitalLife	4u	12u	16u
TransportAI	5u	16u	21u
SemaforoCtrl	8u	15u	23u
EnergyGrid	9u	17u	26u
Média	-	15u	21,5u

Tempo médio de espera (15u) abaixo do limite crítico (16u). Synthetica está salva.

"Falha crítica confirmada. Iniciando protocolo de desligamento de emergência."

5

Observe que **Ana** chegou depois de **Carlos**, mas por ser prioritária ocupa o índice 0. **Pedro** chegou depois de **Beatriz**, mas foi inserido logo após **Ana** — na posição 1. **Carlos**, **Beatriz** e **Silvia** mantêm a ordem em que chegaram, na zona comum.

Você deverá adaptar o código da lista duplamente encadeada implementada em aula para inserir as operações obrigatórias a seguir:

- Para o funcionamento da aplicação, gere um menu de interação com o usuário da aplicação. A aplicação deve permitir que o usuário insira novos pacientes, faça o atendimento, exiba a lista de pacientes, busque um paciente pelo nome e encerre a aplicação (quando o usuário quiser).